

板金見積システム（STEP・DXF・図面PDFからの見積）

設計書

版	2.0
日付	2026-10-05
対象読者	開発者
文書番号	DOC-02

本資料の顧客名・図番・金額はすべて架空のデータです。

目次

1. この資料について	2
2. システム構成	2
3. 主な処理の流れ	3
4. STEPの解析	3
4.1 処理の段階	4
4.2 公差	4
4.3 板厚の認識	5
4.4 表裏の面と曲げの認識	5
4.5 展開（グラフによる展開）	5
4.6 自己検算と状態の決め方	6
4.7 理由コード	8
5. 展開図DXFの解析	8
6. 図面PDFの読み取り	10
6.1 流れ	11
6.2 ページの準備	11
6.3 LLM に渡すもの・返させるもの	12
6.4 ルールでの値の決め方	12
6.5 要確認の決め方	12
7. 見積の計算	13
7.1 明細の計算式	13
7.2 小計・特急割増・粗利・最終価格	14
7.3 見積書を発行できる条件（画面の見積）	15
7.4 確認が必要な点（画面の「確認のヒント」、API の estimate_reasons）	15
8. 見積書・納品書・請求書	16
9. 類似見積・似た図面	16
10. API	17
11. データ設計	18
11.1 データベース	18
11.2 マスター（data/）	19
11.3 見積履歴（data/past_quotes/history.csv）	19
11.4 保存するファイル（output/、Git管理外）	20
12. 評価の仕組み	21
13. LLMを使う箇所	22
14. 主なモジュール	22

1. この資料について

本システムを **どう作っているか** を、実装のロジック（処理の順番、判定の条件、計算式、しきい値）を中心に書く。コードを読まなくても、どの値がどう決まるかが分かることを目指す。何ができるべきかは「要件定義書」、画面は「画面設計書」、評価の結果は「評価報告書」を見る。

数字の出典は `analysis/` の評価レポート、しきい値の出典はソースコード（ファイル名を併記）である。

2. システム構成

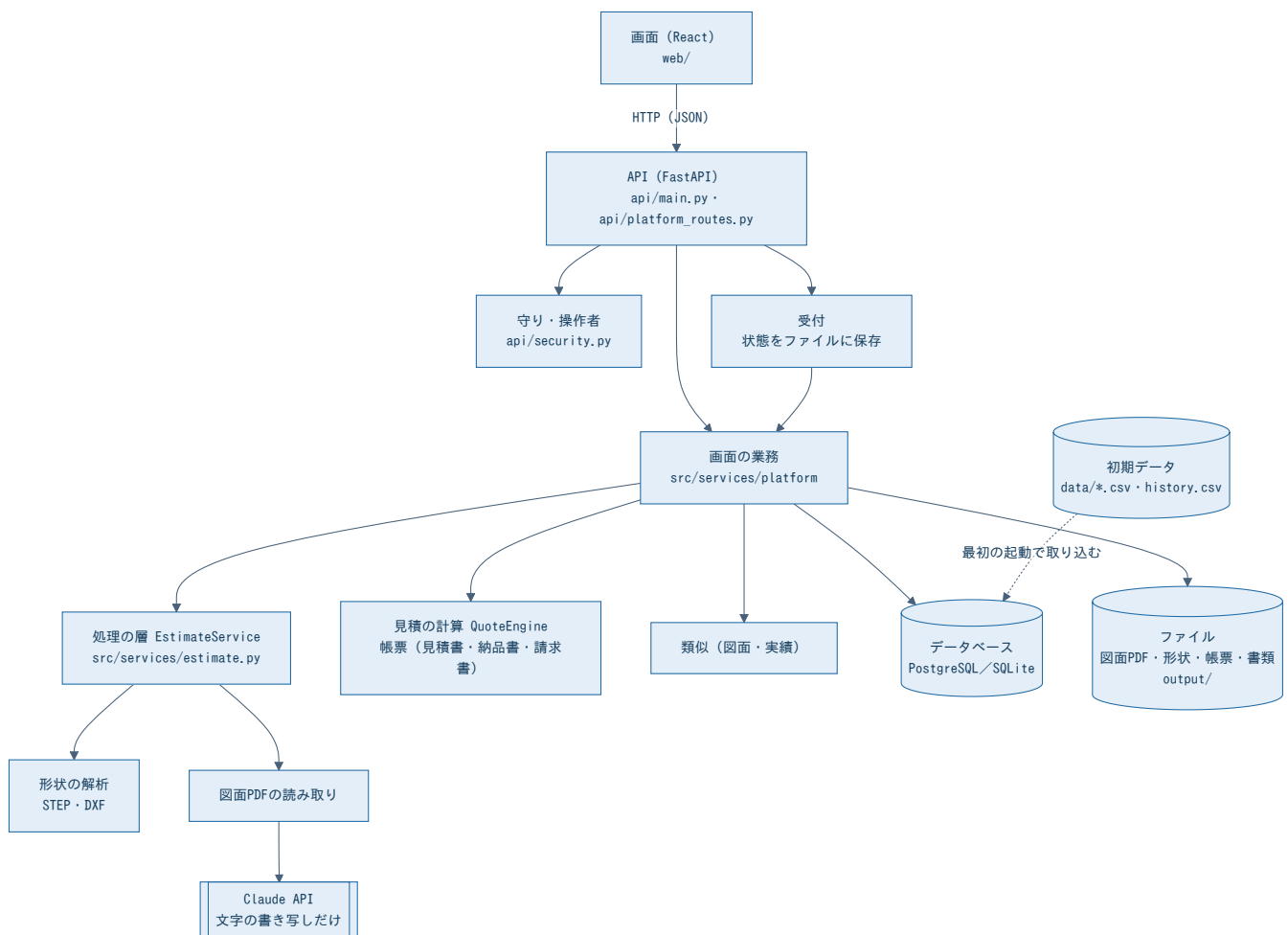


図 システム構成

- 画面は API だけと呼ぶ。金額・未入力的项目・発行できるか・類似かどうかは API（`src/services/platform`）が決め、画面は表示するだけ。処理を2か所に書かない
- LLM を呼ぶのは図面PDFの読み取りだけ（今の読み取り器をそのまま呼ぶ）。金額は `QuoteEngine` と端数処理の規則だけで決まる
- データはデータベースに、ファイル（図面PDF・形状ファイル・帳票PDF・書類）は `FileStore`（`output/`、`ESTIMATE_STORAGE_DIR` で変更）に保管する。マスターと過去見積は CSV を初期データとして取り込み、評価ハネスと既存のテストは今までどおり CSV を直接読む

- 置き方：docker compose up で db (PostgreSQL) ・ api ・ web (nginx) の3つが起動する。社内サーバーでもクラウドの仮想マシンでも同じ

3. 主な処理の流れ

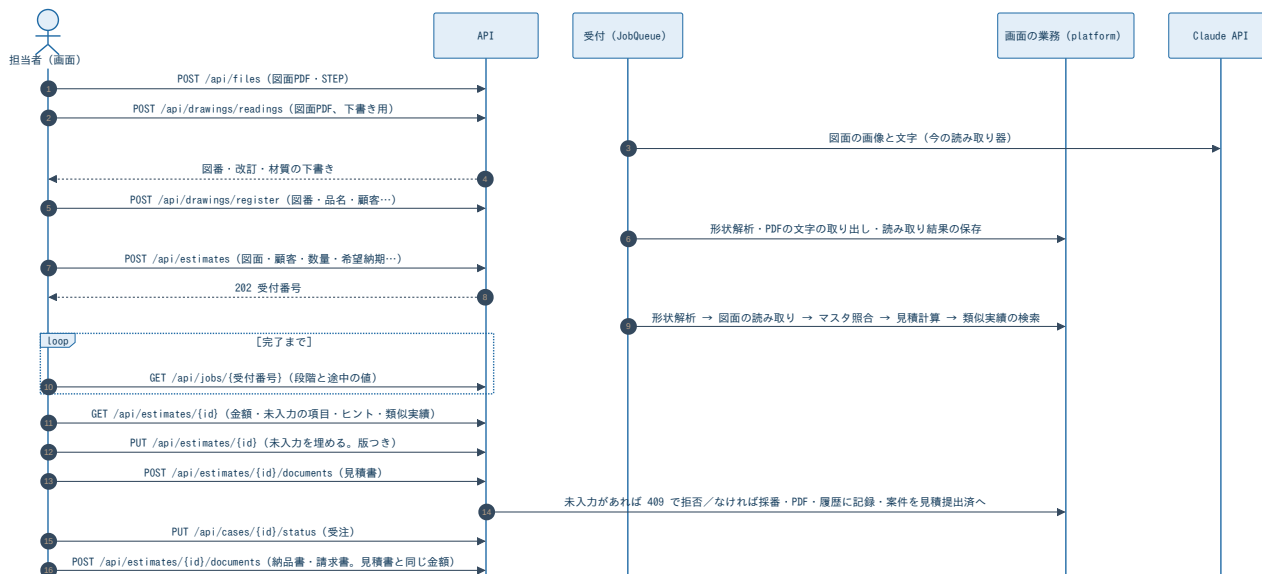


図 図面の登録から見積書の発行まで

4. STEPの解析

src/sheetmetal_analyzer.py (司令塔)、sheetmetal_recognition.py (板厚・表裏・曲げ)、sheetmetal_unfold.py (展開)、sheetmetal_geometry.py (公差・面隣接グラフ)。3D の処理には CadQuery (OpenCASCADE) を使う。LLM は使わない。

4.1 処理の段階

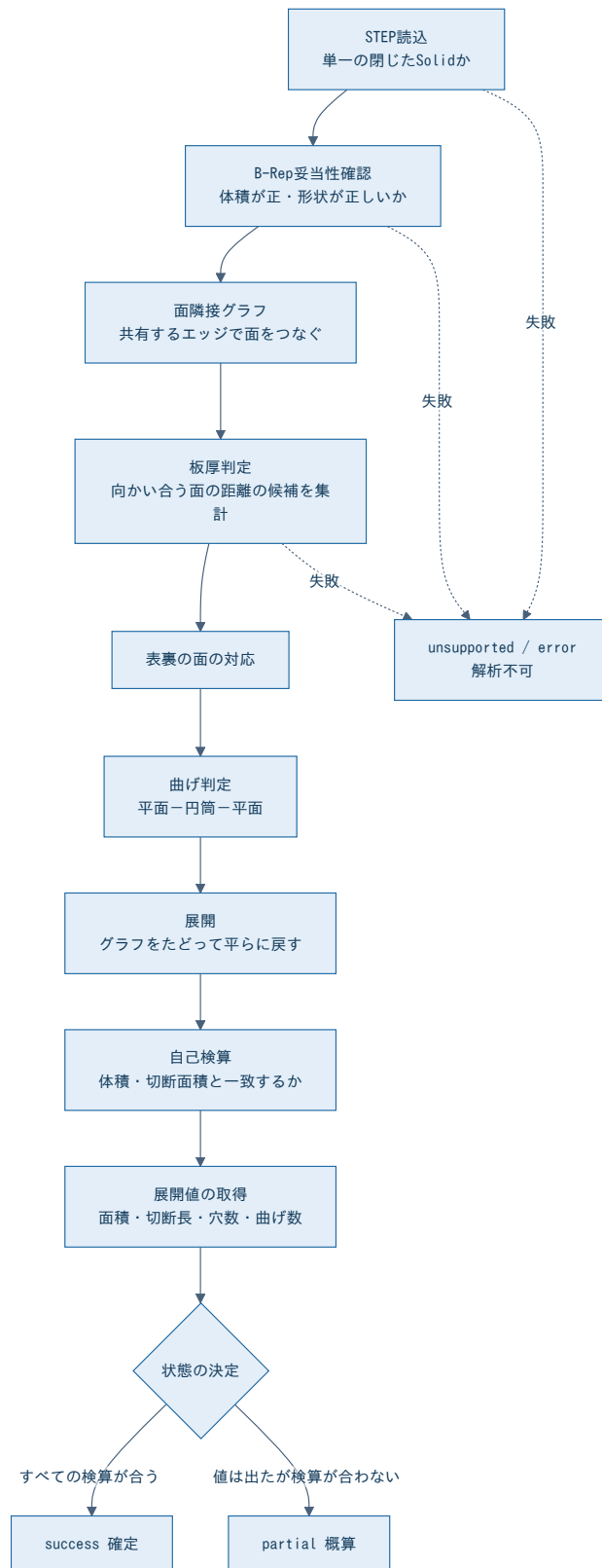


図 STEPの解析の段階（それぞれで失敗したら理由コードつきで止まる）

4.2 公差

部品大きさに合わせて公差を決める（`sheetmetal_geometry.py`）。対角の長さ D (mm) に対し、長さの公差 = $\min(0.01, \max(1e-6, D \times 1e-7))$ mm、角度の公差 $1e-4$ 。

4.3 板厚の認識

1. 向かい合う平面（法線が逆向き、内積が $-1 + 1e-4$ 以下）の組と、同じ軸の円筒の組について、面の間の距離を板厚の候補とする。平面の距離は解析的に計算し、間に材料がない組（ヘムのすき間）は除く
2. 候補を、面積（支持）の重みで集計し、相対差 2% 以内をまとめる（`THICKNESS_CLUSTER_REL_TOLERANCE` = 0.02）。最も支持の大きい値を板厚とする
3. 次の検査に通らなければ解析不可：
 - 板として薄いか：板厚 $\div \sqrt{(\text{体積} \div \text{板厚})}$ が 0.20 を超えたら板金でない（`MAX_SHEET_SLENDERNESS`、`NOT_SHEET_METAL`）
 - 表裏の対応で得られる面積と、体積 $\div$ 板厚 の差が 8% を超えたら、板厚が一定でない（`AREA_CONSISTENCY_TOLERANCE`、`NON_CONSTANT_THICKNESS`）

4.4 表裏の面と曲げの認識

- **表裏の面の対応**：距離が板厚に一致（相対 2.5% 以内、`THICKNESS_MATCH_REL_TOLERANCE`）する向かい合った平面の組、同じ軸で半径の差が板厚の円筒の組を「表裏のペア」とする
- **曲げ**：平面－円筒－平面 の隣接関係から曲げを見つける。円筒の軸が曲げ軸、内側の半径が内R、外側が外R。曲げ角度 $\theta = \text{内側の円筒の面積} \div (\text{内R} \times \text{軸方向の長さ})$ 。170° 以上をヘムとして報告する
- 曲げを決められないときは `BEND_UNDETERMINED`（概算）

4.5 展開（グラフによる展開）

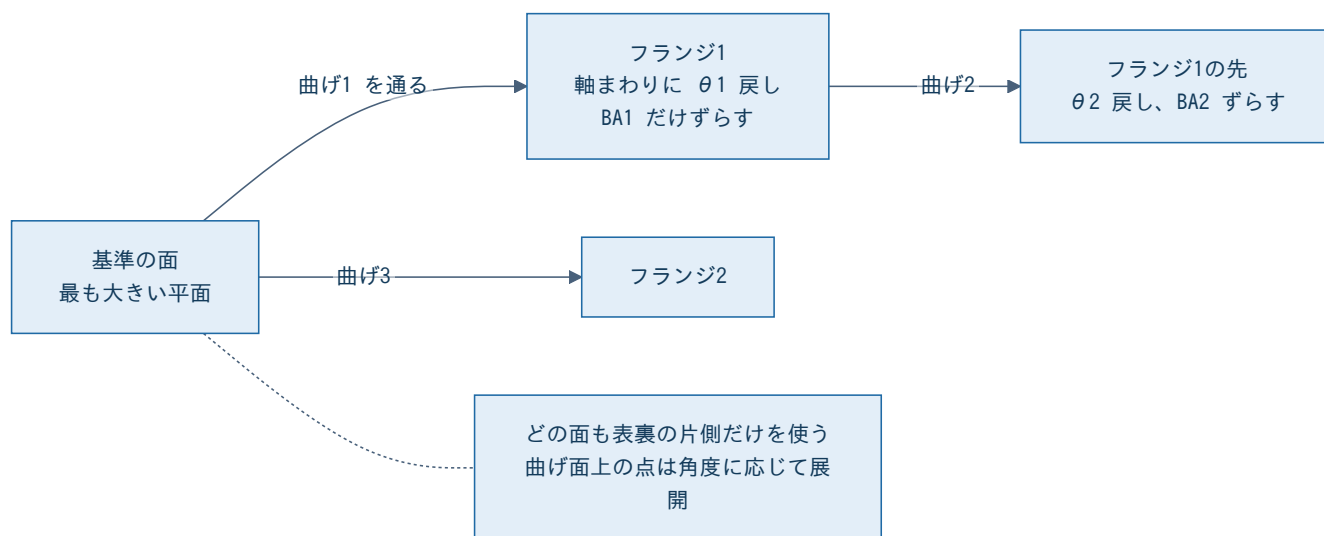


図 展開の考え方：面の隣接グラフをたどり、曲げを通るたびに平らに戻す

- 最も大きい平面を基準にし、表裏のうち片側の面（平面と曲げ円筒）だけを、隣接グラフで幅優先にたどる
- 曲げを通るたびに、その先の面を曲げ軸まわりに円筒の角度だけ戻し、曲げ代 $BA = \theta \times (\text{内R} + K \times t)$ だけ平行移動する（ θ はラジアン、 t は板厚、 K はKファクター）。これは折り曲げの厳密な逆変換で、平板・平行曲げ・トレー・多段フランジ・ヘムを同じ処理で扱う
- 片側の表面の境界エッジを、共有する頂点でつないで展開図の閉じた輪郭（ループ）を作る
- **展開面積** = 外側のループの面積 - 内側のループ（穴）の面積
- **切断長** = 境界エッジの長さの合計（平面上のエッジは長さが変わらない。曲げの端は展開後の長さ）
- **穴数** = 内側のループの数

Kファクターは、加工条件として指定されていない場合は既定値 0.33 で計算し、前提（assumptions）に記録して信頼度を medium（見積りは概算）にする。

4.6 自己検算と状態の決め方

展開の結果を、展開とは別の方法で求めた値と比べる（`sheetmetal_analyzer.py`、`sheetmetal_unfold.py`）。

検算	式	許容差
面積	展開面積 = 体積 ÷ 板厚 + $\sum \theta \times t \times (K - 0.5) \times$ 曲げの長さ	1% (<code>AREA_CHECK_TOLERANCE</code>)
切断長	切断長 = 切断面（板厚方向の面）の面積 ÷ 板厚 + $\sum$ 曲げの端ごとの $\theta \times t \times (K - 0.5)$	1% (<code>CUT_CHECK_TOLERANCE</code>)
展開の整合	すべての表裏ペアのちょうど片側が展開に入っている。展開後に平らである。閉じた経路で位置がずれない	—
輪郭	展開図が1つの単純な外形で、穴は外形の内側にあり、互いに重ならない	—
数	切断面のまとまりの数 = 展開図のループの数。すべての曲げが展開に入っている	—

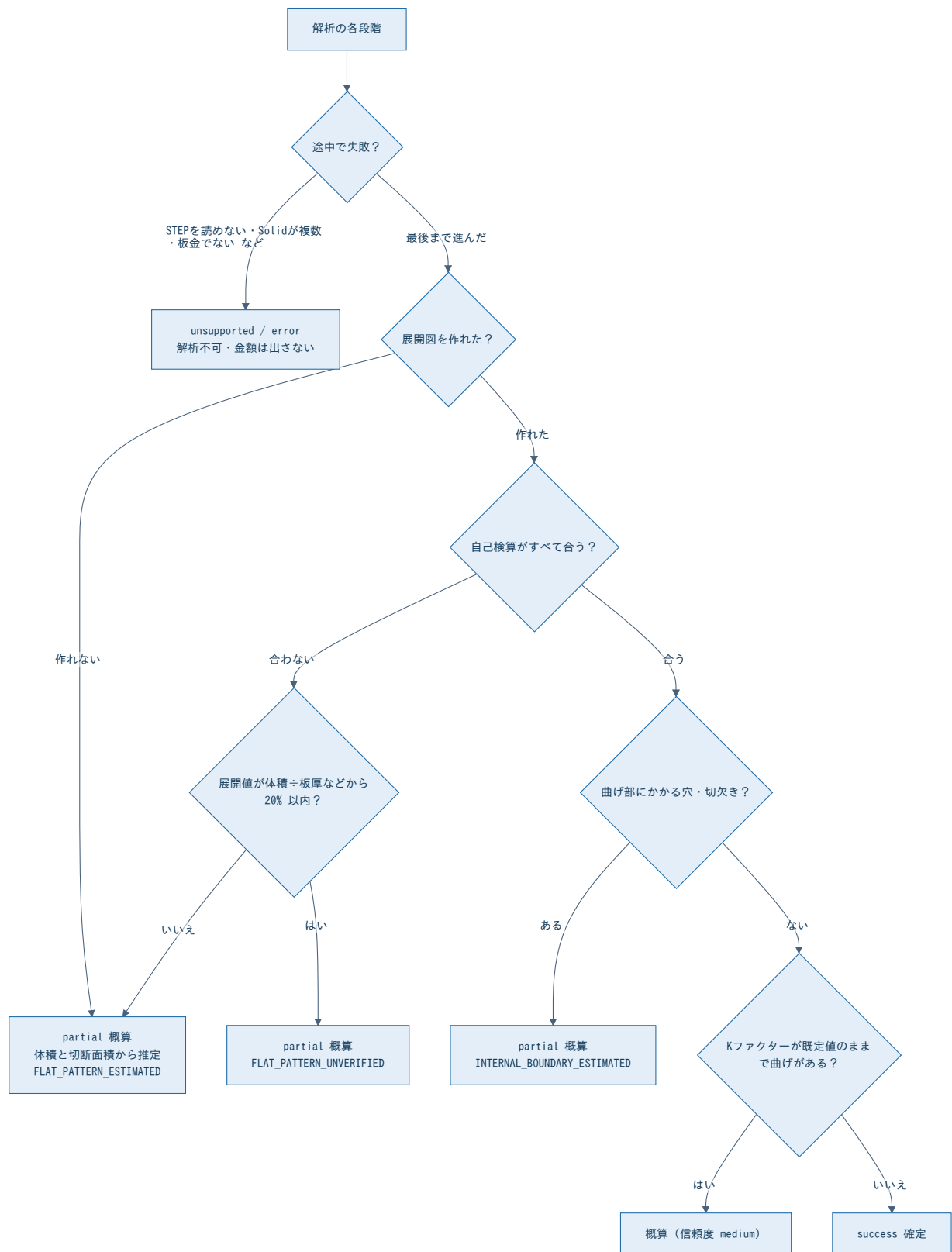


図 結果の状態の決め方

検算が合わない展開値は、**物理的な概算**（面積 = 体積 ÷ 板厚 + K の補正、切断長 = 切断面の面積 ÷ 板厚）と比べ、どちらかが 20% を超えて外れていれば使わず、物理的な概算を返す（ESTIMATE_BOUND = 0.20、FLAT_PATTERN_ESTIMATED、信頼度 low）。閉じた筒のように展開の経路が一周してずれる形で、面積がほぼ0や負の値のまま材料費になるのを防ぐ。面積が切断長が0以下になるときは解析不可（FLAT_PATTERN_FAILED）にする。

4.7 理由コード

理由コード	意味	状態
INVALID_FILE_TYPE ・ STEP_READ_ERROR	STEP でない・読めない	解析不可
NO_SOLID ・ MULTIPLE_SOLIDS	Solid がない・複数ある（アセンブリ）	解析不可
BREP_INVALID	形状データが正しくない	解析不可
NOT_SHEET_METAL ・ NON_CONSTANT_THICKNESS	板金でない・板厚が一定でない	解析不可
ANALYZER_UNAVAILABLE	CadQuery が入っていない	解析不可
BEND_UNDETERMINED	曲げを決められない	概算
FLAT_PATTERN_UNVERIFIED	展開図を作ったが自己検算が合わない	概算
INTERNAL_BOUNDARY_ESTIMATED	曲げ部にかかる穴・切欠きがある	概算
FLAT_PATTERN_ESTIMATED	展開図を作れない、または検算が合わない展開値が物理的な概算から20%超外れたため、体積と切断面積から推定	概算
FLAT_PATTERN_FAILED	切断面がない、面積・切断長が正にならない	解析不可

5. 展開図DXFの解析

src/dxf_analyzer.py（DxfAnalyzer）。ezdxf と shapely によるルールベース。**板厚は入力**（DXF には板厚がない）。LLM は使わない。

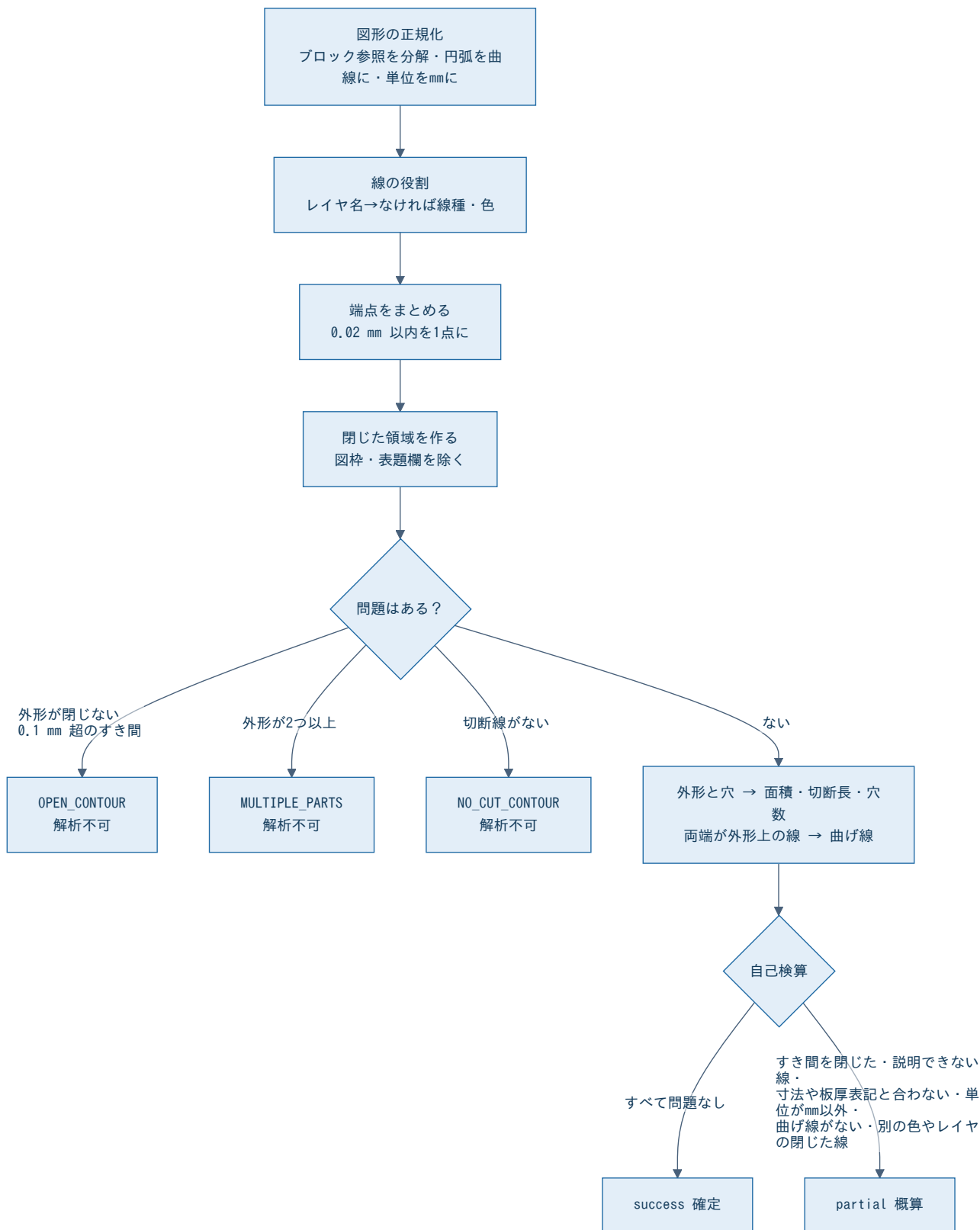


図 展開図DXFの解析の流れ

段階	ロジック
図形の正規化	ブロック参照 (INSERT) を分解する (ブロック内のレイヤ0・BYBLOCK の属性は参照側から継承)。LWPOLYLINE の円弧 (bulge) ・ARC・CIRCLE を曲線にする。\$INSUNITS で mm に換算する (単位なしは mm とみなし概算。mm 以外の単位も概算: mm で描いた図面の単位がインチになっていると値が25.4倍になり、ファイルの中に見分ける手がかりがないため、UNIT_NOT_MM)。DIMENSION・TEXT は形状として扱わず、寸法に 表示された数値 (上書きの文字、なければ寸法ブロックの文字 ÷ DIMLFAC) と板厚表記は検算に使う (ezdxf が形状から計算した寸法値は、形状と比べても意味がないので使わない)
線の役割	レイヤ名に意味があれば使う (CUT/外形、BEND/曲げ線、DIM/寸法、FRAME/図枠、CENTER/中心線、MARK/ケガキ など英日の表記ゆれ)。なければ線種で判断し、実線は切断線の候補、実線以外の直線は曲げ線・中心線の候補
端点をまとめる	端点どうしの距離が 0.02 mm 以内なら1点にまとめる (SNAP_MM。格子への丸めは使わない: すき間が格子の境目をまたぐと閉じないため)。重複線は1本にし、重複した曲線の間にできる幅 0.05 mm 未満の細い面は無視する
部品の特定	連結した線のまとまりごとに閉じた領域を作る。複数の面に分かれ、ほかの図形を囲み、 図を囲む面のほかの面 (表題欄の枠) が隅にまとまっている (幅・高さとも全体の90%未満に収まる) まとまりを図枠・表題欄として除く。実線の曲げ線で帯に分かれた部品は、帯が部品の幅いっぱいに渡るので図枠にしない。残った閉じた輪郭のうち最も外側を外形、その内側を穴とする。外形と 別の色 の閉じた線は穴に数えない (ケガキ・文字枠の可能性)、 別のレイヤ の閉じた線は穴に数える。どちらも概算にする
問題の検出	外形が閉じない (0.1 mm を超えるすき間、NEAR_GAP_MM) → OPEN_CONTOUR、外形になり得る輪郭が2つ以上 → MULTIPLE_PARTS、切断線がない → NO_CUT_CONTOUR (いずれも解析不可)
曲げ線	途中で分割された線をつないだうえで、両端が外形上にあるものを曲げ線とする。穴の中心で交差する線は中心線として除く。曲げ線が1本もないときは、平板と曲げ線の描き漏れを区別できないので概算 (NO_BEND_LINES)。利用者が平板と指定したとき (画面の「曲げなし (平板)」、API の flat_confirmed) だけ確定にできる
自己検算 (概算にする条件)	0.02~0.1 mm のすき間を閉じた (SMALL_GAP_CLOSED)、部品の内側に外形と同じ色の閉じない線がある (UNEXPLAINED_GEOMETRY。別の色ならケガキ線として除外)、役割の分からない破線がある (UNEXPLAINED_LINE)、寸法に表示された数値が外形の大きさと合わない (差が max(0.1 mm, 0.5%, 表示の桁の半分) を超える、DIMENSION_MISMATCH)、表題欄の板厚表記と入力した板厚が違う (THICKNESS_MISMATCH)、単位が mm 以外 (UNIT_NOT_MM)、曲げ線がない (NO_BEND_LINES)、別の色・レイヤの閉じた線 (UNEXPLAINED_GEOMETRY)

6. 図面PDFの読み取り

src/pdf_reader.py (PdfConditionReader)、src/pdf_terms.py (語の解釈ルール)、src/pdf_quote.py (見積への反映)。LLM (Claude) は「図面に書いてある文字の書き写し」だけに使い、コード・数値・確定かどうかはルールで決める。

6.1 流れ

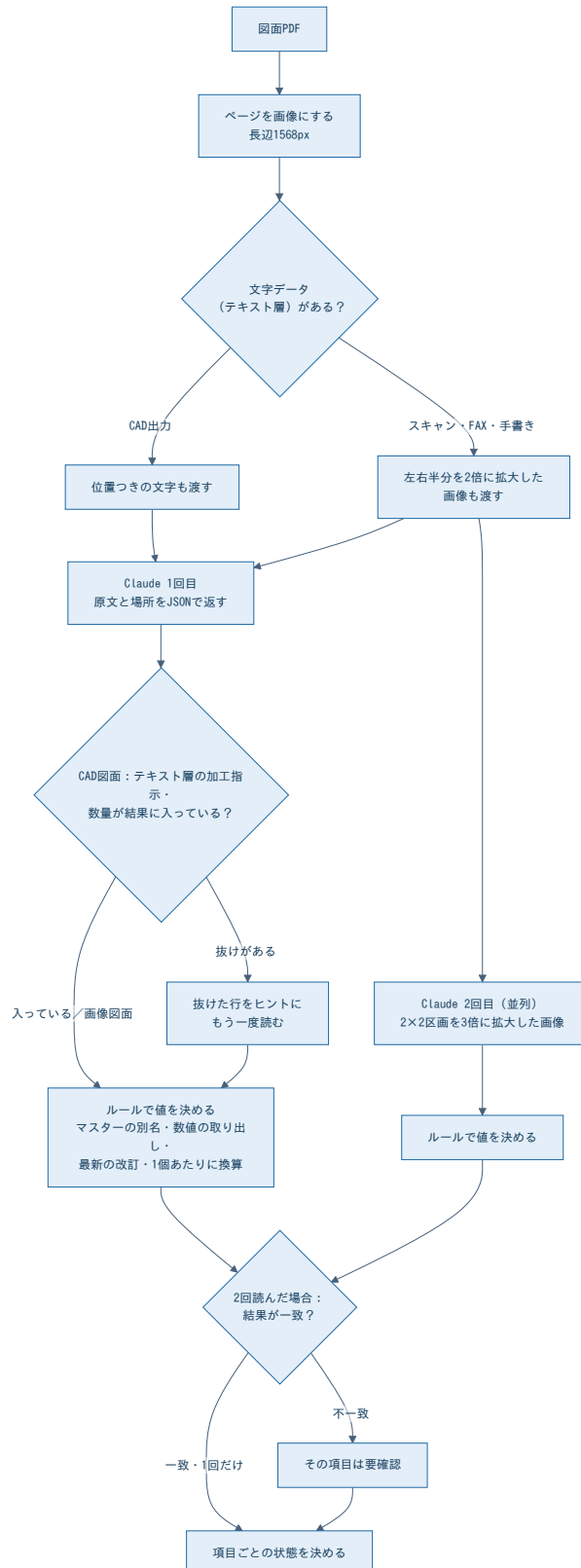


図 図面PDFの読み取りの流れ

6.2 ページの準備

- ページ全体を長辺 1568 px の画像にする（LONG_EDGE）
- スキャン・FAX・手書きの図面と A3 は、左右半分を2倍に拡大した画像を加える

- CAD出力の図面は、文字データ（テキスト層）を、左上からの位置（mm）つきで取り出して一緒に渡す
- スキャン・FAX・手書きの2回目の読み取りには、ページを 2×2 の区画に分けて3倍に拡大した画像を使う

6.3 LLM に渡すもの・返させるもの

- システムプロンプトの要点：値の原文だけを書き写す／改訂表・手書きの訂正・追記を反映した最新の値を書く／空欄や書かれていない項目は null／ただの穴は追加加工ではない／分けて書かれた指示は別々に、拡大画像で重複して見えるものは1回だけ／部品表の行は数量と見出し（台分・合計など）も／特急と特記事項の規則／手書き・押印はすべて注記として書く（手書きの「至急」は印字の「通常」より優先）／マスターにない表記は「未登録」、似たコードに寄せない
- 出力の形（ツール呼び出しの JSON スキーマで固定）：図番・改訂・改訂履歴（改訂・項目・変更前・変更後）・材質／板厚／数量／表面処理（それぞれ原文・書かれている場所・LLM の解釈）・追加加工の一覧（原文・場所・部品表の数・手書きの追記か・コード・数）・部品表の数量の見出し・特急（値と原文）・特記事項（区分と原文）・手書きと押印の注記・自信のない項目
- モデルは `claude-sonnet-5`（環境変数 `ANTHROPIC_MODEL` で変更可）。形式が正しくない応答は最大2回まで聞き直す。応答は `.claude_cache/` に保存し、同じ問い合わせは再利用する

6.4 ルールでの値の決め方

項目	ルール
材質・表面処理	原文からラベル（「材質：」など）を除き、マスターの別名（ <code>aliases</code> ）と表記ゆれの規則でコードにする。完全一致で決め、似たものに寄せない。マスターにないものは <code>UNREGISTERED</code> （未登録）。表面処理の欄に材質名が書かれていれば表面処理は記載なし
板厚・数量	原文から数値を取り出す（「t1.6」「2.3mm」「50個」など）。改訂の変更（「数量 50→100」）があれば最新の値
追加加工	タップ（4-M4、M4タップ 4ヶ所、4X M4x0.7 TAP など）・皿もみ・圧入ナット／スタッド・スポット溶接・TIG溶接をコードと個数にする。ただの穴（φ4.5 キリ など）は数えない。同じ加工は合計し、部品表の合計（「〇台分」「合計」）は数量で割って1個あたりにする。ルールと LLM のコードが食い違えば要確認
特急	「特急」「至急」「URGENT」などで特急。否定（「急ぎません」「特急不要」「特急：なし」「特急 ×」「特急扱い不要」「NO URGENT」「NOT URGENT」「URGENT: NO」など）は特急でない。原文の語と LLM の判断が食い違えば語で決め、要確認にする。手書き・押印が印刷と食い違う（印刷は「急ぎません」で手書きが「至急」など）ときは手書き・押印の値を表示し、要確認にする
特記事項	厳しい公差・外観の指定・検査や証明書の要求。普通公差・定型注記・色の指定は含めない。CAD図面ではテキスト層からもルールで拾い、2回読みでは両方の結果を合わせる

6.5 要確認の決め方

次のどれかに当たる項目は「要確認」にする。画面では読み取った値が入力欄に入り（金額に入る）、担当者が図面と見比べて違えば直す。API（`POST /api/quotes`）では、確定の項目だけを金額に入れる。

- CAD図面：値の原文がテキスト層に見つからない。テキスト層にある加工指示が結果にない（ヒントつきで読み直しても欠ける）
- スキャン・FAX・手書き：2回の読み取りの結果が一致しない
- ルールと LLM の判断が食い違う、改訂の最新値と本文の値が違う、原文をルールで解釈できない、部品表の合計が数量で割り切れない、手書き・押印の特急の指示が印刷と食い違う

LLM が自分で「自信がない」と返した項目（`uncertain`）は判定に使わない（評価で、正しい値にも多く付き、自動確定を下げるだけだったため）。

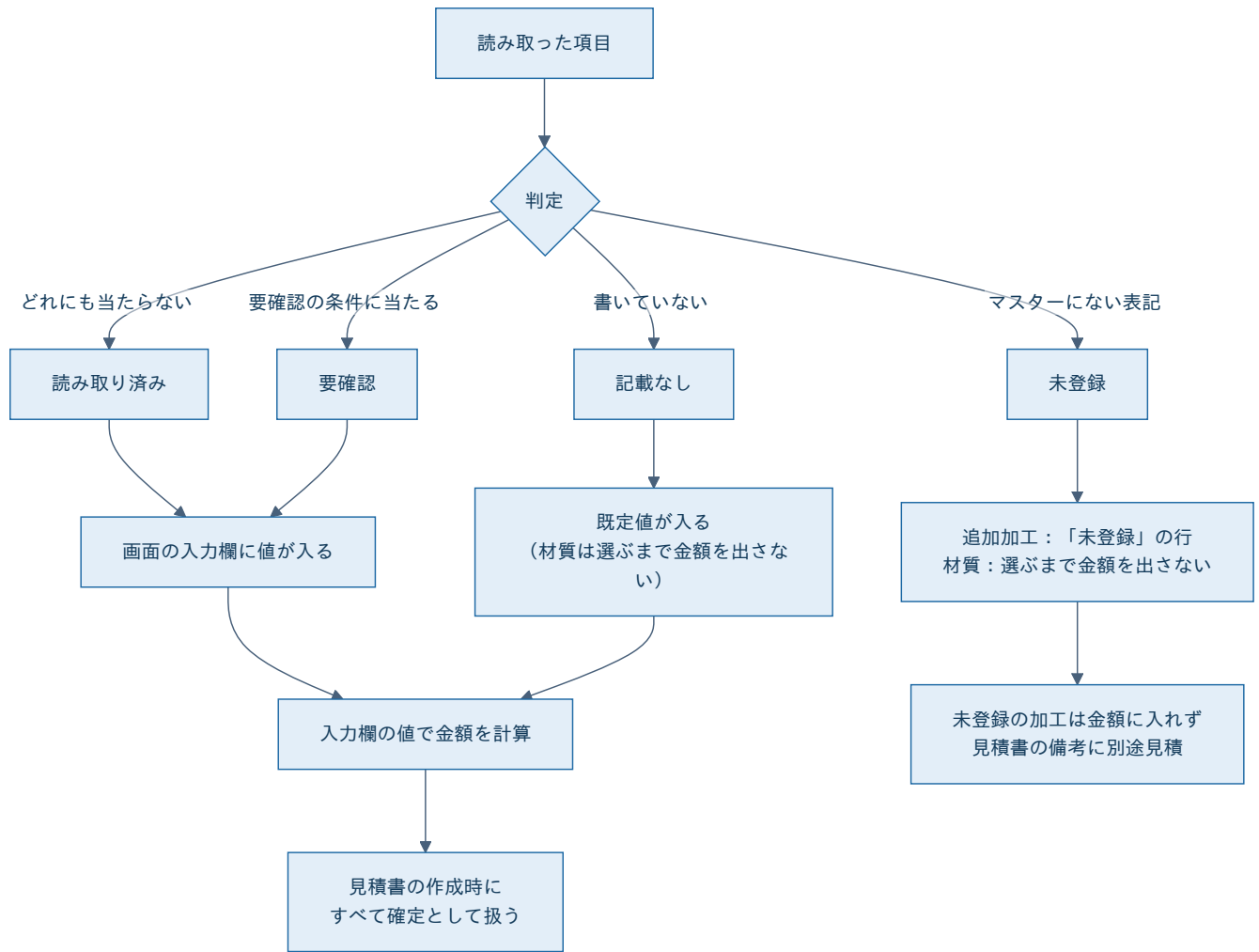


図 項目ごとの状態（画面の表示と見積への反映）

7. 見積の計算

`src/quote_engine.py`（QuoteEngine）。マスターの単価と率だけで計算する。

7.1 明細の計算式

数量を N 、展開面積を A (mm^2)、板厚を t (mm) とする。

明細	式
材料費	1個の重さ (kg) = $A \times t \div 10^9 \times \text{密度 (kg/m}^3\text{)}$ 。金額 = 重さ $\times$ 材料単価 (円/kg) $\times$ 歩留まり係数 (waste_factor) $\times N$
レーザー切断	切断長 (mm) $\times$ 単価 (円/mm) $\times N$
ピース（穴あけ）	穴数 $\times$ 単価 $\times N$
曲げ	曲げ数 $\times$ 単価 $\times N$
段取り	単価 (1案件に1回)
追加加工	1個あたりの個数 $\times$ 単価 $\times (N \text{ または } 1)$
表面処理	単価 $\times (N \text{ または } 1)$

`charge_scope` が `per_part` の行は数量倍、`per_order` の行は1案件に1回だけ数える（どちらかはマスターの各行で決める）。

7.2 小計・特急割増・粗利・最終価格

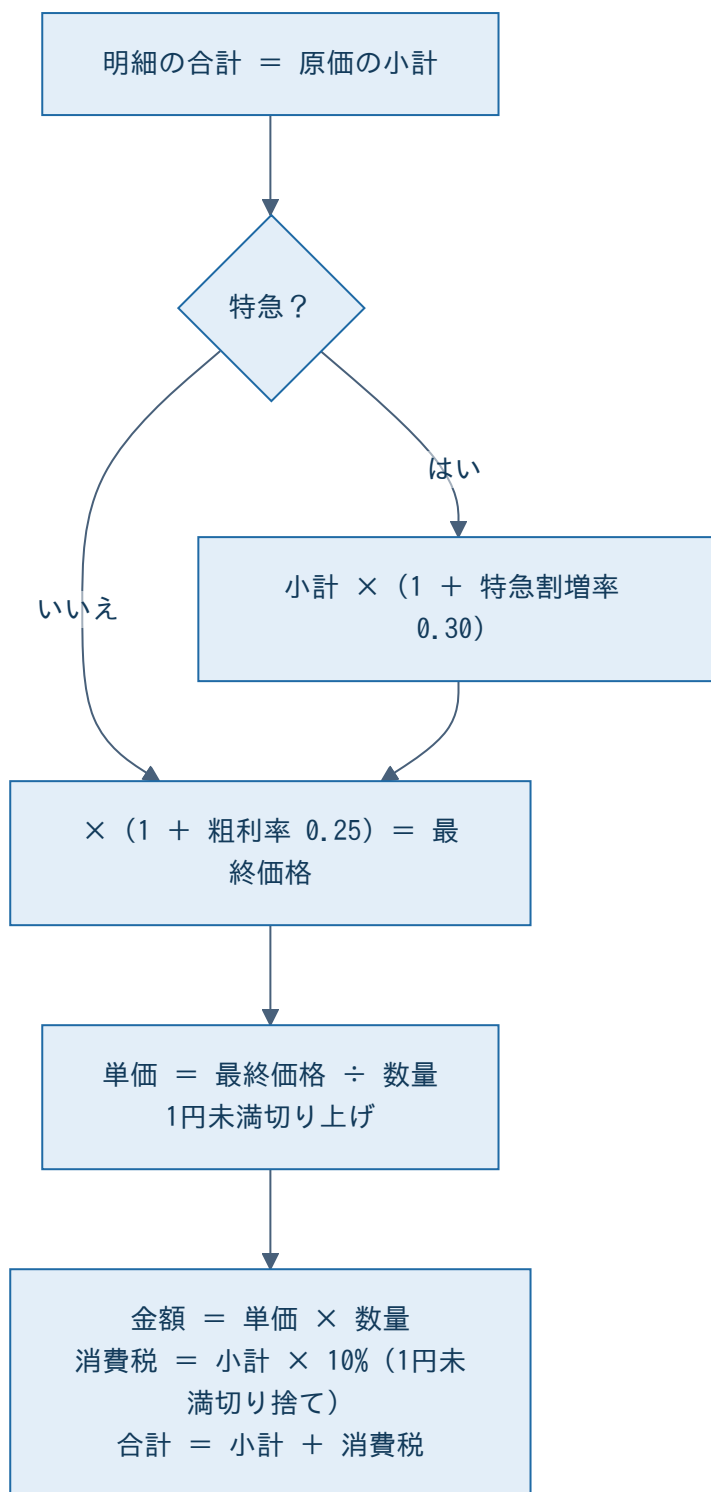


図 見積の計算の順番

- 特急割増率 `rush_surcharge_rate` (0.30)、粗利率 `margin_rate` (0.25)、税率 `tax_rate` (0.10) は `data/pricing_policy.csv`

- 画面の見積は、図面から読み取った値を初期値にした **見積の条件**（`QuoteInputs`、`src/services/platform/pricing.py`）で計算する。読み取りの状態はヒントで、要確認でも値はそのまま使う。板厚は形状の値（STEPから測った板厚、DXFは解析に使った板厚）を使い、図面の板厚は照合だけに使う
- API の `POST /api/quotes`（画面を使わない呼び出し）に図面の項目（`drawing`）を渡すと、**確定の項目だけ**を金額に入れる（今までどおり）。`condition` に数量・表面処理・特急を指定すると、図面の値に代わって確定になる。図面とチャット・手入力の同じ追加加工は、手入力の個数で置き換える
- 単価・金額・消費税・合計の端数処理は、画面・API・帳票で共通の関数（`src/quote_document.py` の `price_summary`）で行う。費目ごとの小計も API が返す。画面・API・帳票PDFで金額が1円も変わらないことをテストで確かめている

7.3 見積書を発行できる条件（画面の見積）

`src/services/platform/pricing.py` の `missing_items`。未入力の項目が1つでもあれば、API が帳票の発行を拒否する（409 `NOT_ISSUABLE`、`details` に一覧）。画面のボタンを無効にするだけではない。

未入力の項目	条件
数量	数量が空
材質	マスターの材質が選ばれていない（マスターにない材質を選んだときは、kg単価と密度がない）
マスターにない表面処理	1個あたりの単価がない
追加加工	マスターの加工の箇所数がない、マスターにない加工の単価（1か所あたり）か箇所数がない
形状の値	板厚・展開面積・切断長・穴数・曲げ数のうち、形状解析で求められず（解析不可・形状ファイルなし）入力もないもの。解析中は「形状解析がまだ終わっていません」

- 図面の読み取りが要確認、形状解析が概算（`partial`、前提、信頼度 `medium/low`）、表面処理「なし」・特急「なし」は、未入力ではない（ヒントとして表示）
- マスターにない材料・加工・表面処理**：似たものに寄せない。担当者が入れた単価で、その見積だけのマスターの写し（`with_custom`）を作り、`QuoteEngine` で計算する。材料はコード＝名前、密度と kg単価は入力値、歩留まり係数はマスターの板材と同じ 1.15、1個ごと。加工は `CUSTOM_P1` …（個数×単価、1個ごと）、表面処理は `CUSTOM_FINISH`。マスターには登録しない
- 形状の値の入力**：解析の値があるものは解析の値を使い、ないものだけ入力値を使う。解析不可・形状ファイルなしのときは、入力値で解析結果（`status partial`、前提「形状の値は担当者の入力」）を作って計算する。内訳に「手入力」とは出さない
- 希望納期**は記録・見積書の備考（「ご希望納期」）・進捗の警告に使い、金額には使わない

7.4 確認が必要な点（画面の「確認のヒント」、API の `estimate_reasons`）

次のどれかに当たると、その点を一覧で示す（`QuoteEngine.is_estimate` は評価で「概算表示」として使う）。見積書は、作成した時点で条件を確定として扱うので、見積書には「概算」の区別はない。

- 解析結果が `partial`、前提（Kファクターの既定値など）がある、信頼度が `medium/low` の値がある
- 図面から読み取った条件に、確定でない項目が残っている（API で確定にしていない項目）、マスター未登録の追加加工がある
- 図面の板厚と形状の板厚が違う

8. 見積書・納品書・請求書

`src/quote_document.py`（中身と金額）、`src/quote_pdf.py`（見積書の描画）、`src/trade_pdf.py`（納品書・請求書）、`src/services/platform/documents.py`（発行）、`src/quote_log.py`（API の見積書の番号）。LLM は使わない。

- ・ **記載項目**（見本 `analysis/quote_document_sample.pdf` と同じ並び）：タイトル・見積番号・発行日・有効期限／宛先（御中・様）／自社（会社名・住所・TEL/FAX・登録番号・担当者・押印欄）／件名・納期・受渡場所・取引条件・有効期限／御見積金額（税込）／明細（No・品名と図番・仕様2行・数量・単位・単価・金額）／小計・消費税・合計／備考
- ・ **タイトル**：つねに「御見積書」。発行した時点で条件を確定として扱う。概算の見積書はない
- ・ **仕様の2行**：1行目「材質 板厚 表面処理」、2行目「レーザー切断・曲げn箇所・追加加工と個数」
- ・ **納期**：「受注後 N営業日」。通常は `lead_time_days_normal`（10）、特急は `lead_time_days_rush`（5）。有効期限は発行日＋`quote_validity_days`（30）
- ・ **備考**：見積の根拠（図面の図番・改訂、形状ファイル名）、自社の定型文、ご希望納期、特急、特記事項（検査成績書などは別途）、追加の備考
- ・ **番号**：画面からの見積書は案件番号（Q-YYYY-NNNN、データベースの連番。再発行は -2 …）、納品書 D-YYYY-NNNN、請求書 I-YYYY-NNNN。API の `POST /api/documents` は Q＋発行日＋－＋連番3桁（`output/quote_log.csv`）
- ・ **テンプレート**：表示する項目（費目の内訳＝QuoteEngine の明細の名前と数量だけで金額は出さない、単価と数量、図番と改訂、有効期限、備考欄、社印）を切り替える。指定がなければ、顧客に当てたテンプレート、なければ既定。原価・粗利は出さない（固定）
- ・ **納品書・請求書**：受注（`outcome` が受注）した案件で、発行済みの見積書の金額（発行記録 `issued_documents` に保存した単価・小計・消費税・合計と明細）をそのまま使う。請求書は適格請求書として、自社の登録番号と税率別の合計（税率・対象金額・消費税・合計）を載せ、支払期限は翌月末
- ・ **プレビュー**：発行するのと同じ PDF を番号「（プレビュー）」で作り、1ページ目を画像にして返す
- ・ **社外用と社内用**：社外用には原価の内訳と粗利率を出さない。API の見積書では社内用（見積根拠、別PDF、社外秘）も出せる
- ・ **日本語**：IPAゴシック（`fonts/ipag.ttf`）を埋め込む。長い文字は縮小し、それでも入らなければ折り返す
- ・ 発行した見積書は見積履歴（データベースの `past_quotes`）に入り（案件ごとに1行。再発行で置き換え）、案件の受注・失注に合わせて結果も変わる。類似実績・振り返り分析に使う

9. 類似見積・似た図面

`src/similar_quotes.py`（規則・過去見積の説明と価格の比較）、`src/services/platform/similar.py`（図面と実績の候補）。LLM は使わない。

- ・ **規則**（`is_similar`）：材料が同じ（マスターのコード。マスターにない・不明の材質は何とも似ない）、曲げ数の差が1以内、穴数の差が2以内。曲げ数・穴数が分からないもの（形状ファイルがない図面、形状の数値がない過去見積）は対象外
- ・ **並べ方**（`difference`）：曲げと穴の差の合計、曲げの差、穴の差の小さい順。同じなら新しい順
- ・ **候補**：登録済みの図面（最新の版の材質と形状解析の曲げ数・穴数、最新の見積の単価・数量・状態）と、登録図面に結び付かない過去見積の履歴。自分の図面は除く。最大30件
- ・ **同じ結果**：図面一覧の「似た形」（`GET /api/drawings/{id}/similar`）、図面詳細、見積結果の類似実績（`GET /api/estimates/{id}` の `similar`）は同じ関数を呼ぶ。見積の材質・形状の値が図面と同じなら同じ結果になる（テストで確認）

- **過去見積の説明**（POST /api/similar-quotes）：区分（リピート＝同じ顧客・同じ図番／同じ顧客／他の顧客）は表示だけで、並び順は上の規則。似ている理由・今回との違い・価格の比較（今のマスターで計算し直した標準単価と出し値の比、今回の標準単価に当てた水準、リピートで±20%を超えると注意）。今回の単価は変えない

10. API

api/main.py（FastAPI、create_app）と api/platform_routes.py（画面用）。仕様書は起動後の /docs、詳しい設計は analysis/api_design.md。

機能	エンドポイント
状態・選択肢	GET /api/health（トークン不要）、GET /api/meta、GET /api/recent
図面	POST /api/drawings/register、GET /api/drawings（条件）、GET /api/drawings/same-number、GET/PUT /api/drawings/{id}、PUT /api/drawings/{id}/current-revision、GET /api/drawings/{id}/similar、POST /api/drawings/{id}/memos、POST /api/revisions/{id}/notes、GET /api/revisions/{id}/preview.png
見積	POST /api/estimates（受付番号）、GET /api/estimates、GET/PUT /api/estimates/{id}、POST /api/estimates/{id}/chat
帳票	GET /api/estimates/{id}/documents/preview.png、POST /api/estimates/{id}/documents、GET /api/issued、GET /api/issued/{id}/file.pdf
案件	GET /api/cases、PUT /api/cases/{id}/status、GET /api/cases/{id}/history、GET/PUT /api/statuses、GET /api/review
検索・書類・取引先	GET /api/search、GET/POST/DELETE /api/library、GET/POST/PUT /api/partners
設定・マスター	/api/staff、/api/categories、/api/attributes、/api/templates（/preview.png）、/api/master-tables/{table}、GET /api/logic、POST /api/history/columns
単体の処理（今までのAPI）	POST /api/files、POST /api/analyses、POST /api/drawings/readings、GET /api/jobs/{job_id}、POST /api/quotes、POST /api/documents、POST /api/similar-quotes、/api/history...、GET /api/files/{file_id}/model.glb、GET /api/masters

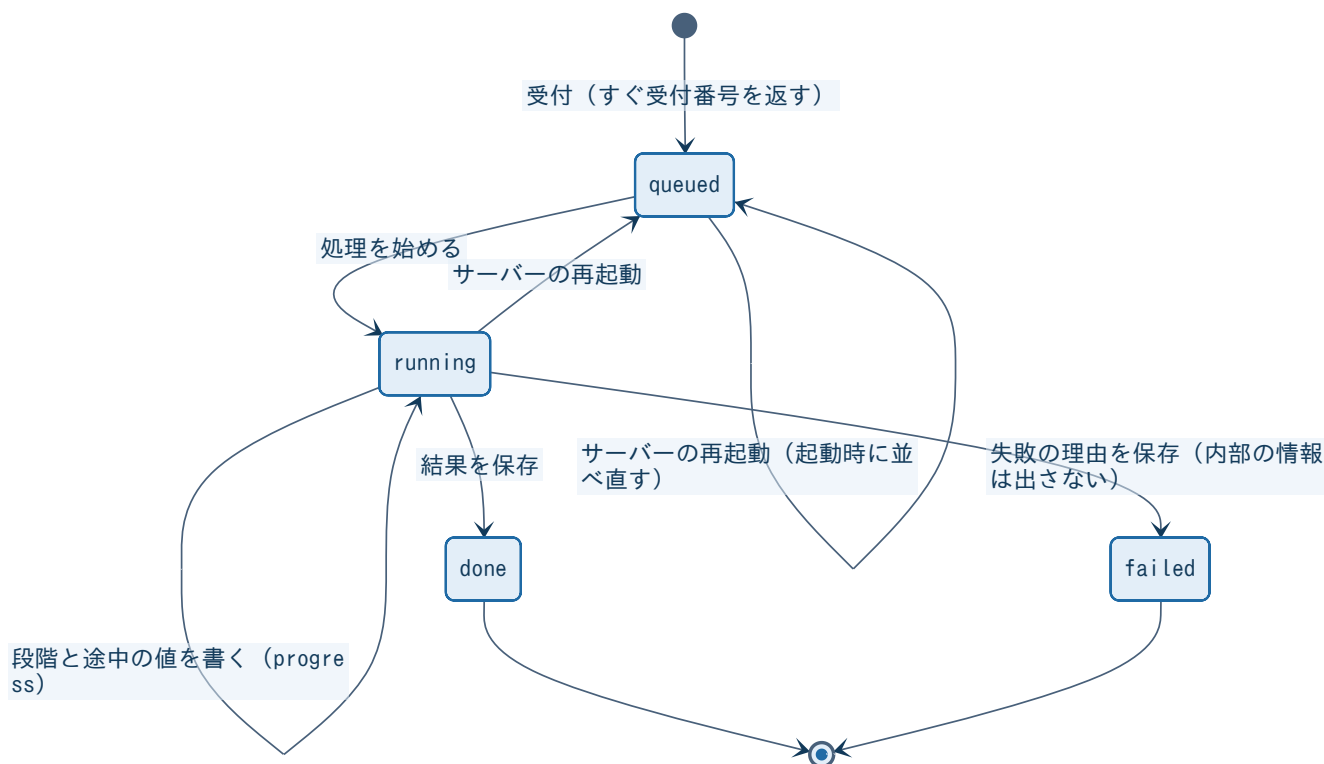


図 受付（解析・読み取り・登録・見積）の状態の移り変わり

- 受付ごとに `<保存先>/jobs/<受付番号>.json` に状態・入力・結果・進み具合を書く。再起動しても消えない。処理は同じプロセスのスレッド（`JOB_WORKERS`、既定2）。pdfium は同時に使えないので、PDF を開く処理は1つの排他（`src/services/pdfium_lock.py`）の下で行う（読み取り器のページの読み込みもこの排他で包む。LLM の呼び出しは並行のまま）
- **同時利用**：見積・案件・図面・取引先・マスター・ステータスの行は版（`version`）を持つ。更新は開いたときの版を送り、違えば `409 CONFLICT`（後から保存した側に知らせ、上書きしない）。データベースの側でも版を照合して更新する（SQLAlchemy の `version_id_col`）。連番は行ロックで取る
- **守りと操作者**（`api/security.py`）：合言葉（`API_TOKEN`）を全エンドポイント（`health` を除く）に1か所で掛ける。操作者は画面で選んだ担当者（`X-Actor`）で、修正履歴・ステータスの履歴・帳票の発行・登録に残す。ログインと権限を足すときは、この2つの関数を差し替える
- 守り：アップロードの種類（拡張子と中身の先頭）と大きさ（`MAX_UPLOAD_MB`、既定50）、エラーに内部のパスを出さない

11. データ設計

11.1 データベース

`src/db/models.py`（SQLAlchemy 2）。構造の変更は Alembic のマイグレーション（`migrations/versions/`）で管理し、API の起動時に `upgrade head` する。`DATABASE_URL` がなければ `<保存先>/app.db`（SQLite）。初期データ（`src/db/seed.py`）は、空の表にだけ入れる。

表	中身
<code>drawings</code> ・ <code>drawing_revisions</code>	図面（図番・品名・顧客・自社の属性・最新の版）と版（改訂・図面PDFと形状ファイルの <code>file_id</code> ・材質・表面処理・解析結果・読み取り結果・最大寸法・PDFの文字）
<code>drawing_categories</code> ・ <code>categories</code>	図面と分類（グループとフォルダ）の多対多
<code>drawing_notes</code> ・ <code>drawing_memos</code>	図面への書き込み（版ごと、位置 0~1）、注意・不具合メモ
<code>cases</code> ・ <code>case_status_log</code>	案件（番号・顧客・案件名・ステータス・担当・希望納期・結果・失注の理由と他社価格・各日時）と、ステータスの履歴（当時の名前）
<code>quotes</code> ・ <code>quote_edit_log</code>	見積（図面と版・条件 <code>inputs</code> ・解析・読み取り・計算結果・チャット）と修正履歴（だれが・何を・金額の前後）
<code>issued_documents</code>	発行した帳票（種類・番号・金額・テンプレート・ファイル・明細）
<code>past_quotes</code>	見積履歴（CSV と同じ列と、案件・図面・失注の理由・回答日数）
<code>master_materials</code> ・ <code>master_processes</code> ・ <code>master_surface_treatments</code> ・ <code>master_pricing_policy</code> ・ <code>company</code>	マスターと自社情報（CSV と同じ列。文字のまま持ち、 <code>MasterLoader</code> と同じ形で <code>QuoteEngine</code> に渡す）
<code>case_statuses</code> ・ <code>attribute_defs</code> ・ <code>templates</code> ・ <code>staff</code> ・ <code>partners</code> ・ <code>documents</code>	案件ステータス（色・グループ・表示・役割）、属性項目、帳票テンプレート、担当者、取引先、書類（文字データ・関連図面）
<code>counters</code>	連番（案件・納品書・請求書）と、マスター・履歴の変更回数（読み込み直しの合図）

11.2 マスター（data/）

ファイル	列	意味
<code>materials.csv</code>	<code>material</code> , <code>display_name</code> , <code>density_kg_m3</code> , <code>price_per_kg</code> , <code>waste_factor</code> , <code>charge_scope</code> , <code>aliases</code>	材質コード、表示名、密度、材料単価（円/kg）、歩留まり係数、数え方、表記ゆれ（ 区切り）
<code>process_rates.csv</code>	<code>process_code</code> , <code>display_name</code> , <code>calculation_type</code> , <code>unit_price</code> , <code>unit</code> , <code>charge_scope</code> , <code>aliases</code>	工程コード（LASER_CUT・PIERCE・BEND・SETUP と追加加工）、単価、単位（mm・hole・bend など）
<code>surface_treatments.csv</code>	<code>treatment_code</code> , <code>display_name</code> , <code>unit_price</code> , <code>charge_scope</code> , <code>aliases</code>	表面処理コード（NONE＝処理なし）、単価
<code>pricing_policy.csv</code>	<code>key</code> , <code>value</code> , <code>description</code>	<code>margin_rate</code> 0.25、 <code>rush_surcharge_rate</code> 0.30、 <code>tax_rate</code> 0.10、 <code>quote_validity_days</code> 30、 <code>lead_time_days_normal</code> 10、 <code>lead_time_days_rush</code> 5
<code>company.csv</code>	<code>key</code> , <code>value</code> , <code>description</code>	自社情報（仮の値）。 <code>company.local.csv</code> （Git管理外）があればそちらを使う。 <code>remark_1</code> … が備考の定型文

表記ゆれの照合は `MasterLoader.resolve_alias`（全角→半角、大文字、空白や記号を除いて完全一致）。似たものへの寄せはしない。

11.3 見積履歴（data/past_quotes/history.csv）

1行1見積の UTF-8 CSV。コミットして、Git で変更が読める。最初の起動でデータベースの `past_quotes` に取り込み、以後はデータベースで持つ（ファイルは書き換ええない）。

列	意味
quote_no, date, customer, drawing_no, revision, part_name	見積番号・見積日・顧客・図番・改訂・品名
material_text / material_code / material_family	書かれた文字、マスターのコード（系統だけ分かるときは空、マスターにないときは UNREGISTERED）、系統（steel・stainless・aluminum・copper）
thickness, quantity	板厚・数量
finish_text / finish_code	表面処理の文字とコード（空欄は「記録なし」）
processes_text / processes	追加加工の文字と、JSON の一覧（コード・個数・原文）
rush, unit_price, amount, outcome	特急、単価、金額、結果（受注・失注・未回答）
area, cut, holes, bends, flat_size, shape_class	形状の数値（ない見積は空）
source, original	取り込み元（import:ファイル名 か app）、取り込んだ元の行（JSON）

過去見積の CSV（Excel の書き出し、UTF-8 か Shift_JIS）は、列名の別名（「顧客名／得意先」など）が明示した対応づけで取り込む。同じ見積（番号・日付・顧客が同じ）は二重に入らない。

11.4 保存するファイル（output/、Git管理外）

場所	中身
output/app.db	データベース（DATABASE_URL がないとき。SQLite）
output/uploads/<file_id>/	アップロードしたファイル（図面PDF・形状ファイル・帳票PDF・書類）と meta.json
output/jobs/<job_id>.json	受付の状態・入力・結果
output/documents/<見積番号>/	見積書（quote.pdf）と社内用（internal.pdf）
output/viewer/	3D表示用の形状（glb）、図面PDFのプレビュー画像
output/quote_log.csv	見積書の出力記録

置き場所は環境変数で変えられる（ESTIMATE_DATA_DIR、ESTIMATE_STORAGE_DIR、PAST_QUOTES_PATH、QUOTE_LOG_PATH）。

12. 評価の仕組み

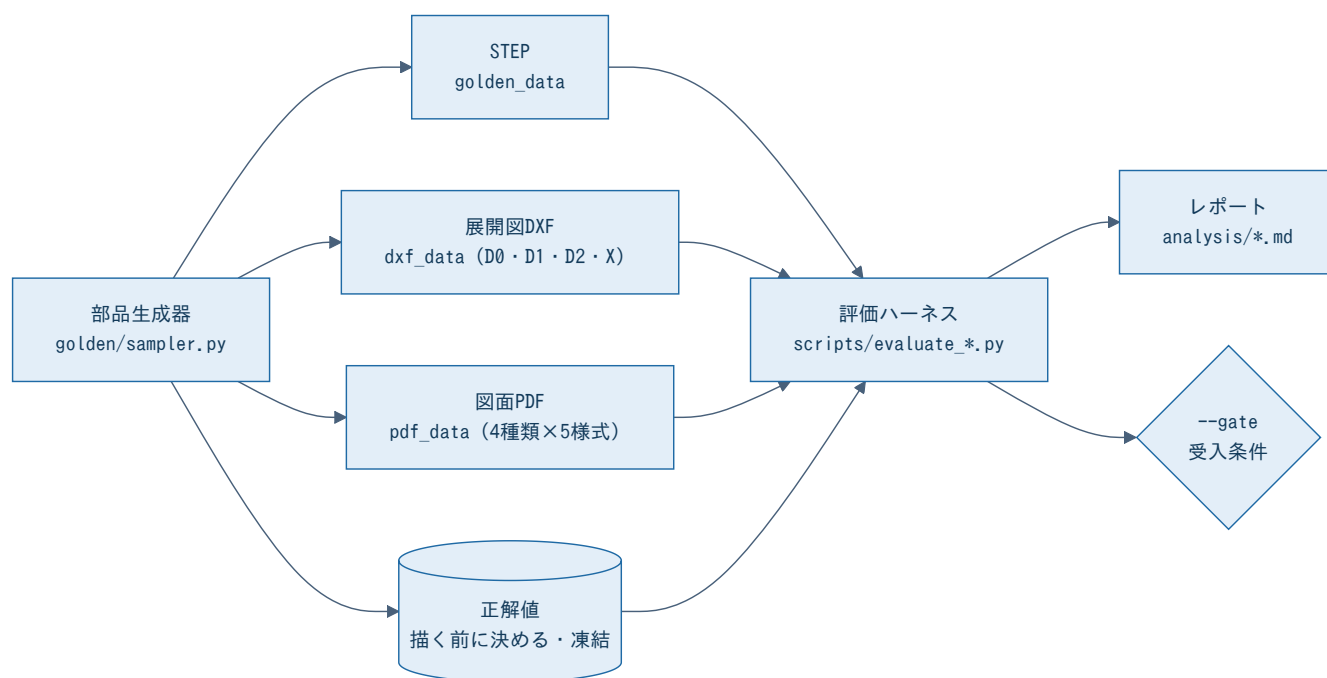


図 評価の仕組み

- ・ **ゴールデンデータ**：正解の分かる部品を部品生成器で作り、STEP・DXF・図面PDFに描き分ける。正解値は描く前に決め、凍結したファイル（golden/datasets/*.json）と照合する（--verify-frozen）
- ・ **評価ハーンズ**：scripts/evaluate_golden.py（STEP）、scripts/evaluate_dxf.py（DXF）、scripts/evaluate_pdf.py（図面PDF）。部品ごとの結果は */results.csv、集計は analysis/*.md
- ・ **判定の区分**：確定で正解／正解だが概算／誤り（概算表示あり）／危険誤答（誤っているのに確定表示）／解析不可。図面PDFは項目ごとに 正解／正解だが要確認／誤り（要確認）／危険誤答
- ・ **ホールドアウト**：開発中は見ない評価用データを別の乱数（seed）で作り、最後に一度だけ評価する。作り込みすぎ（開発用データへの過剰な適合）を見つけるため

評価の結果（analysis/ のレポートから）：

対象	データ	結果
STEP	開発用508件・ホールドアウト200件・追加検証2,000件	すべて「確定で正解」100%、危険誤答0件
DXF	開発用800・ホールドアウト400・追加検証800ファイル	解析成功100%（X は全件「確定しない」）、危険誤答0件。開発用は規則の追加後に再評価し、確定で正解は D0・D1 80%、D2 58%（残りは平板・インチ単位の図面で、正解だが概算）
図面PDF	開発用300枚	価格項目がすべて正しい99.3%、自動確定92.7%、危険誤答0.0%（ホールドアウトは判定途中）
類似見積	種データ1,045件	平均誤差 全体10.8%→7.9%、リピート9.9%→4.9%

13. LLMを使う箇所

箇所	入力	出力	金額・数値への使い方
図面PDFの読み取り (Claude)	図面の画像、CAD図面の文字データ、システムプロンプト	項目ごとの原文と書かれている場所 (JSON)	原文をルールでコード・数値にする。LLMの解釈はルールとの照合（食い違えば要確認）にだけ使う
条件のチャット入力 (OpenAI、任意)	担当者の文章、登録済みの工程の一覧	変更の操作（数量・材質・工程の追加など）	登録済みのコードと数量だけを反映（図面の条件の入力欄にも入る）。APIキーがなければ簡易的な解釈
LLM単独の解析器（比較用、既定では使わない）	STEP の面の一覧	板厚・面積など	結果は常に概算。評価の比較にだけ使う

使わない箇所：STEP・DXFの形状の解析、見積の計算、帳票、類似、検索、API の処理。図面の登録で AI の仕分けはしない。これらは LLM のキーがなくてもすべて動く。

14. 主なモジュール

モジュール	役割
web/	画面（React・TypeScript、Vite で web/dist に作る）
api/main.py ・ api/platform_routes.py ・ api/security.py	API（FastAPI）、画面用の API、守りと操作者
src/services/	処理の層（estimate.py ・受付 jobs.py ・型 schemas.py ・設定 settings.py ・3D gltf.py ・pdfium_lock.py）
src/services/platform/	画面の業務（pricing.py ・quotes.py ・drawings.py ・cases.py ・documents.py ・similar.py ・library.py ・admin.py ・masters.py ・core.py）
src/db/ ・ migrations/	データベースのモデル・接続・初期データ、マイグレーション
src/sheetmetal_*.py	STEPの解析
src/dxf_analyzer.py	展開図DXFの解析
src/pdf_reader.py ・ pdf_terms.py ・ pdf_quote.py	図面PDFの読み取りと見積への反映
src/quote_engine.py	見積の計算
src/quote_document.py ・ quote_pdf.py ・ quote_log.py ・ trade_pdf.py	見積書・納品書・請求書
src/past_quotes.py ・ similar_quotes.py	見積履歴と類似見積
data/	マスター・自社情報・見積履歴（初期データ。評価とテストはこの CSV を読む）
scripts/seed_demo.py ・ tests/test_e2e_flow.py	お試しデータ、ブラウザでの通しの操作のテスト
golden/ ・ scripts/evaluate_*.py	ゴールデンデータの生成と評価